

CLAUDE.md — PodClick project context

Standing rules for every session. For the current milestone's step-by-step, see [docs/CLAUDE_TASK.md](#). For full detail, see [docs/PodClick_Studio_Descript_Feature_Spec.md](#) and [docs/PodClick_Master_Architecture_Spec.md](#).

Prime directive

Enhance, don't rebuild. PodClick Studio already works. Do not redesign existing surfaces, do not move or rename existing controls, do not regenerate a parallel version of something that exists. Add to what's here.

Before building any feature, map it against [docs/PodClick_Master_Architecture_Spec.md](#) and tag it  (exists) /  (partial) /  (missing). Reuse the existing clip reframer, caption styling, two-speaker layouts, and render/QC pipeline rather than duplicating them.

What the product does (two jobs)

1. **Direct-to-podcast/clip/YouTube** — capture/upload → Publish → existing audio + AI pipeline (Job Site). Automated. Already built.
2. **Single video** → **Descript-style edit** → **repurpose** — capture/upload → lands in the Video Library as MP4 → edit → export. Being added.

Both share capture, upload, and the **Video Library**. The Library item is the fork between the two jobs.

Information architecture (non-negotiable)

- The transcript editor is a **separate area of the UI**, opened from a **Video Library item** ("Open in Editor"). It is NOT a third mode inside Studio.
- The Studio **Podcast Episode / Direct Video** toggle stays as-is — it only expresses *format intent at capture*. Do not overload it.
- The **workflow fork** (send to pipeline vs. open in editor) happens at the **Library**, not at capture.
- Editor reads/writes Library assets; it does not own separate storage.

Core mental model (the editor)

The transcript IS the timeline. Every word carries a source start/end timestamp; the renderable output (the EDL) is *derived* from the ordered word list. Editing the transcript edits the video. The visual timeline is a projection of the word list, never the primary surface.

Invariants — do not violate

- **Never renumber `Word.seq`.** It is the stable original transcription index the EDL compiler uses to decide source-adjacency. Delete flips a flag; reorder changes array position — `seq` is untouched in both.
- **Every Library asset must have a finite duration** before it can be edited. Captured WebM reports `Infinity/NaN` until repaired (see `media.ts`). Fix at the source; reject non-finite duration.
- All word-list edits go through `editor-core/src/edit.ts`. Do not hand-roll array mutations in the UI.
- After any edit, recompile via `compileEDL(words)` before previewing/exporting.

The verified core — do not rewrite

`packages/editor-core/src/` is tested (`npm test`, expect all passing). Treat it as a dependency, not a draft. Keep the tests green.

- `edl.ts` — `compileEDL` / `validateEDL` (the crux; verified against delete, natural-pause, reorder, cross-media, strikethrough).
- `transcribe.ts` — `Transcriber` interface + `WhisperXTranscriber`.
- `edit.ts` — delete / restore / strikethrough / correct / move ops.
- `media.ts` — WebM duration fix + finite-duration probe.
- `render.ts` — EDL → ffmpeg trim+concat export.

Tech choices (locked unless explicitly revisited)

- **Transcription: WhisperX** — free, self-hosted, word-level. On the iMac use `device: "cpu"`, `compute_type: "int8"`, diarization off. To change engines, implement the `Transcriber` interface; do not touch the editor.
- **Rendering: ffmpeg** (trim+concat filtergraph for arbitrary in/out points).
- Edit operations are **pure functions returning new arrays** (React-friendly, immutable). Keep them that way.

Build discipline

- Ship the cut-and-export loop before any polish: open a Library video → transcribe → edit the word list → `compileEDL` → `validateEDL` → `render`.
- Phases 2–3 (filler/retake/gap tools, scenes/layouts, captions UI, B-roll, audio, transitions, studio sound, voice regenerate, green screen, Underlord) come later and are mostly more pure functions over the same `Word[ ]`. Don't build ahead of the current milestone.